ALGORITMI E STRUTTURE DATI

AULA VIRTUALE DEL 27-10-2026

Esercizio di progettazione

In un sistema informatico che gestisce una piattaforma di e-commerce, è necessario implementare in modo efficiente una struttura dati che mantenga la cronologia degli ordini recenti di un utente.

Il sistema deve supportare:

1. l'inserimento rapido di nuovi ordini in coda alla cronologia;
2. l'accesso diretto a un ordine tramite indice;
3. la crescita dinamica della struttura quando il numero di ordini supera la capacità disponibile;
4. una buona efficienza complessiva anche nel caso di molte inserzioni consecutive.

Quali strutture dati e algoritmi utilizzaresti per progettare una soluzione efficiente motivando il tutto?

In particolare:

- indica quale struttura dati useresti per memorizzare dinamicamente gli ordini;
- descrivi come gestiresti il problema del ridimensionamento;
- spiega perché il costo di alcune operazioni può essere elevato;
- analizza la complessità delle operazioni fondamentali;
- giustifica perché, nonostante alcuni inserimenti costosi, il costo medio rimane efficiente;
- confronta brevemente questa soluzione con un approccio alternativo meno efficiente.

Soluzione proposta

Dati i requisiti dati la struttura dati proposta è un **array dinamico**. L'array dinamico soddisfa in modo naturale questi requisiti, poiché consente accesso diretto in **tempo costante** e inserimento in coda in **tempo costante ammortizzato**.

Struttura dati utilizzata

La struttura è composta da:

- un array che contiene gli ordini;
- una variabile **dimensione**, che rappresenta il numero di ordini effettivamente presenti;
- una variabile **capacita**, che rappresenta il numero massimo di celle attualmente allocate.

L'idea è inserire ogni nuovo ordine nella prima posizione libera dell'array. Quando l'array è pieno, si esegue un'operazione di ridimensionamento.

Gestione del ridimensionamento

Il problema del ridimensionamento si presenta quando il numero di ordini raggiunge la capacità disponibile. In tale situazione, non conviene aumentare la capacità di una sola unità, poiché questo comporterebbe troppe riallocazioni e copie.

La strategia più efficiente consiste nel:

1. creare un nuovo array con capacità doppia;
2. copiare tutti gli elementi dal vecchio array al nuovo;
3. aggiornare il riferimento alla nuova struttura;
4. inserire il nuovo elemento.

Questa strategia riduce il numero di riallocazioni complessive e permette di ottenere un **costo ammortizzato costante** per l'inserimento.

Perché alcune operazioni sono costose

L'inserimento di un nuovo ordine è normalmente molto economico: basta memorizzarlo nella prima cella libera e incrementare la dimensione. In questo caso il costo è **O(1)**.

Tuttavia, se l'array è pieno, prima dell'inserimento è necessario:

- allocare un nuovo array;
- copiare tutti gli elementi già presenti.

Se gli elementi memorizzati sono n, tale operazione ha costo **O(n)**. Di conseguenza, alcune singole operazioni possono essere costose, anche se la maggior parte rimane molto efficiente.

Analisi della complessità

Le operazioni fondamentali hanno la seguente complessità:

- **Accesso a un ordine tramite indice:** O(1)
- **Aggiornamento di un ordine tramite indice:** O(1)
- **Inserimento in coda senza ridimensionamento:** O(1)
- **Inserimento in coda con ridimensionamento:** O(n)
- **Inserimento ammortizzato:** O(1)
- **Scansione completa della cronologia:** O(n)

Questa analisi utilizza gli strumenti di notazione asintotica previsti dal corso e riflette l'obiettivo di confrontare l'efficienza delle soluzioni in termini di tempo e spazio.

Giustificazione dell'efficienza media: analisi ammortizzata

L'aspetto teoricamente più rilevante del problema è l'**analisi ammortizzata**.

Sebbene un singolo inserimento possa costare O(n) nel momento del ridimensionamento, tale situazione si verifica raramente. Se la capacità viene raddoppiata ogni volta che l'array è pieno, i costi di copia si distribuiscono su una lunga sequenza di inserimenti.

Infatti, le copie effettuate nelle varie riallocazioni seguono una progressione del tipo:

$$1 + 2 + 4 + 8 + \cdots + 2^k$$

che è limitata superiormente da 2n. Quindi, eseguendo n inserimenti, il costo totale resta **O(n)** e il costo medio per operazione è:

$$\frac{O(n)}{n} = O(1)$$

Pertanto, si può affermare che l'inserimento in un array dinamico ha costo **O(1) ammortizzato**.

Confronto con una soluzione meno efficiente

Una possibile alternativa meno efficiente sarebbe aumentare la capacità dell'array di **una sola posizione alla volta**. In questo caso, ogni volta che l'array è pieno, si dovrebbe creare un nuovo array di dimensione **capacita** + 1 e copiare tutti gli elementi già presenti.

Il costo complessivo di n inserimenti risulterebbe:

$$1 + 2 + 3 + \cdots + n = O(n^2)$$

Questa soluzione è quindi molto meno efficiente rispetto alla strategia di raddoppio.

Un'altra alternativa sarebbe una **lista concatenata**. Tale struttura permette inserimenti efficienti, ma non consente accesso diretto per indice: per accedere all'elemento in posizione i occorrerebbe scandire la lista, con costo **O(n)**. Poiché il problema richiede esplicitamente accesso diretto a un ordine tramite indice, la lista concatenata non rappresenta la scelta migliore.

Conclusione

In conclusione, la struttura dati più opportuna per mantenere la cronologia degli ordini recenti di un utente è un **array dinamico con ridimensionamento per raddoppio**. Tale scelta consente:

- accesso diretto agli elementi in tempo costante;
- inserimento in coda in tempo costante ammortizzato;
- crescita dinamica efficiente della struttura;
- un buon compromesso tra semplicità implementativa ed efficienza complessiva.

Snippets di codice Python

1. Implementazione essenziale di un array dinamico

```
class CronologiaOrdini:
    def __init__(self):
        self.capacita = 1
        self.dimensione = 0
        self.ordini = [None] * self.capacita

    def inserisci(self, ordine):
        if self.dimensione == self.capacita:
            self._ridimensiona()

        self.ordini[self.dimensione] = ordine
        self.dimensione += 1

    def _ridimensiona(self):
        nuova_capacita = self.capacita * 2
        nuovo_array = [None] * nuova_capacita

        for i in range(self.dimensione):
            nuovo_array[i] = self.ordini[i]

        self.ordini = nuovo_array
        self.capacita = nuova_capacita

    def get(self, indice):
        if indice < 0 or indice >= self.dimensione:
            raise IndexError("Indice fuori intervallo")
        return self.ordini[indice]
```

```
  def aggiorna(self, indice, nuovo_ordine):
    if indice < 0 or indice >= self.dimensione:
      raise IndexError("Indice fuori intervallo")
    self.ordini[indice] = nuovo_ordine

  def __len__(self):
    return self.dimensione
```

2. Versione con conteggio del costo per mostrare l'analisi ammortizzata

```
class CronologiaOrdiniAnalisi:
  def __init__(self):
    self.capacita = 1
    self.dimensione = 0
    self.ordini = [None] * self.capacita
    self.costo_totale = 0

  def inserisci(self, ordine):
    costo_operazione = 1  # costo base di inserimento

    if self.dimensione == self.capacita:
      nuova_capacita = self.capacita * 2
      nuovo_array = [None] * nuova_capacita

      for i in range(self.dimensione):
        nuovo_array[i] = self.ordini[i]
        costo_operazione += 1  # ogni copia aumenta il costo

      self.ordini = nuovo_array
```

```
            self.capacita = nuova_capacita

        self.ordini[self.dimensione] = ordine
        self.dimensione += 1
        self.costo_totale += costo_operazione

        return costo_operazione

    def costo_medio(self):
        if self.dimensione == 0:
            return 0
        return self.costo_totale / self.dimensione
```

3. Esempio di utilizzo

```
cronologia = CronologiaOrdiniAnalisi()

for i in range(10):
    costo = cronologia.inserisci(f"Ordine_{i}")
    print(f"Inserimento {i}: costo = {costo}")

print("Numero ordini:", cronologia.dimensione)
print("Capacità attuale:", cronologia.capacita)
print("Costo totale:", cronologia.costo_totale)
print("Costo medio ammortizzato:", cronologia.costo_medio())
```